

Relatório de Laboratório

Laboratório de Experimentação de Software - Lab02 - Assistentes de IA vs. codificação manual

Relatório final do experimento controlado com 12 execuções, análise estatística e métricas estruturais de código.

Curso	Engenharia de Software
Disciplina	Laboratório de Experimentação de Software
Turno / Período	Noite / 6º
Professor(a)	Daniilo Maia
Laboratório	Lab02 - Assistentes de IA vs. codificação manual
Grupo (trio)	Ana Luiza Santos · Bruna Bernardes · Walter Louback
Link do repositório / GitHub Projects	bruna-bernardes/Lab_Experimentacao_Software https://github.com/users/bruna-bernardes/projects/2/views/1
Data de entrega	25/09/2026

1. Introdução

Assistentes de IA generativa passaram a integrar o cotidiano do desenvolvimento de software, mas seu impacto em produtividade e qualidade depende do contexto de uso e precisa ser avaliado de forma controlada. O Lab02 investiga esse impacto comparando a resolução de tarefas de programação com e sem assistência de IA, mantendo participantes, linguagem, ambiente, limite de tempo e instrumentos de medição sob condições previamente definidas.

O experimento utilizou quatro katas em Python e três estudantes, em desenho crossover/within-subject contrabalanceado. Cada participante resolveu dois katas com GitHub Copilot e dois sem IA, totalizando 12 execuções, seis por tratamento. As Questões de Pesquisa e hipóteses foram:

- RQ1 - O uso de assistente de IA reduz o tempo necessário para resolver uma tarefa? H1: o tratamento COM_IA apresenta menor tempo de resolução.
- RQ2 - O uso de assistente de IA reduz a quantidade de defeitos? H1: o tratamento COM_IA apresenta maior taxa de sucesso e menos testes falhando.
- RQ3 - O uso de assistente de IA altera a complexidade ciclomática ou a duplicação do código? H1: pelo menos uma métrica estrutural difere entre os tratamentos.

Como contribuição adicional correspondente aos 30% de inovação, o grupo registrou métricas exploratórias além do núcleo do enunciado: tempo até o primeiro teste passar, número de iterações de código, quantidade de execuções automáticas dos testes, número de prompts nos trials COM_IA e Maintainability Index. Também foi conduzida uma análise de correlação de Spearman entre LOC, complexidade ciclomática e manutenibilidade.

2. Contexto

O estudo foi desenvolvido ao longo das Sprints Lab02S01, Lab02S02 e Lab02S03. Na Sprint 1 foram definidos o desenho experimental, hipóteses, ameaças à validade, quatro katas, plano de contrabalanceamento e scripts de cronometragem e métricas. Na Sprint 2 foram executadas as 12 tarefas experimentais e preservadas as soluções finais. Na Sprint 3 foram realizadas as análises estatísticas, a análise estrutural da RQ3 e a geração do dashboard.

O tratamento COM_IA utilizou GitHub Copilot no Visual Studio Code, com seleção de modelo em modo Auto. O tratamento SEM_IA proibiu o uso de assistentes generativos. O time-box máximo foi de 35 minutos por execução. Os quatro objetos experimentais foram: Tarifa de Entrega, Cobrança de Estacionamento, Processamento de Pedidos e Classificação de Chamados por SLA.

O planejamento produziu seis execuções COM_IA e seis SEM_IA. Todas as 12 execuções terminaram antes do limite e com 100% dos testes de aceitação passando, portanto não houve observações censuradas.

3. Metodologia

3.1 Principais Desafios

- Manter comparabilidade entre participantes e tratamentos, reduzindo o efeito de diferenças individuais de habilidade.
- Evitar vazamento de soluções entre integrantes e reutilização indevida de implementações previamente vistas.
- Garantir instrumentação uniforme de tempo, testes, complexidade, LOC e duplicação.
- Lidar com amostra muito pequena (3 participantes e 12 execuções), o que reduz o poder estatístico do teste de Wilcoxon.
- Interpretar RQ2 com variância zero, pois todos os trials terminaram com 100% de sucesso.
- Usar LOC como métrica de controle para evitar interpretar verbosidade como complexidade estrutural.

3.2 Tomadas de Decisão

- Adotar desenho crossover/within-subject contrabalanceado, em que cada participante atua como seu próprio controle.
- Usar quatro katas de dificuldade comparável, com 10 testes de aceitação cada e sem dependências externas.
- Usar mediana e IQR na descrição dos resultados, por causa do N reduzido e da presença de valores extremos.
- Aplicar Wilcoxon pareado na análise inferencial, agregando RQ1/RQ2 por participante e RQ3 por kata.
- Usar Cliff's delta como tamanho de efeito, destacando a direção e magnitude mesmo quando o p-valor não alcança 0,05.
- Manter GitHub Projects como fonte de rastreabilidade, com Issues individuais para as execuções e artefatos de cada sprint.

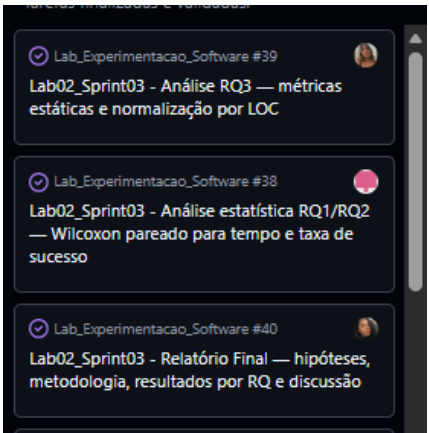


Figura 1 - Recorte do GitHub Projects com atividades da Sprint 3.

3.3 Etapas

Sprint	Entregas	Responsável(is)	Issues
Lab02S01	Cronômetro e coleta das execuções	Bruna	#18
Lab02S01	Desenho e plano do experimento	Ana	#19
Lab02S01	Coleta de métricas estáticas	Walter	#20
Lab02S01	Snapshot do Project	Bruna	#21
Lab02S01	Preparar Katas 01 e 02	Bruna	#22
Lab02S01	Preparar Katas 03 e 04	Ana	#23
Lab02S02	4 execuções de Bruna	Bruna	#24, #25, #27, #28
Lab02S02	Aprimorar coleta com métricas extras	Bruna	#26
Lab02S02	4 execuções de Walter	Walter	#29-#32
Lab02S02	4 execuções de Ana	Ana	#33-#36
Lab02S02	Snapshot do Project	Bruna	#37
Lab02S03	Análise estatística RQ1/RQ2	Walter	#38
Lab02S03	Análise RQ3 e normalização por LOC	Bruna	#39

Lab02S03	Relatório Final	Ana	#40
Lab02S03	Snapshot do Project	Bruna	#41

3.4 Ferramentas

- Python - execução dos scripts de coleta, consolidação e análise.
- Visual Studio Code - ambiente de desenvolvimento.
- GitHub Copilot (Auto) - assistente utilizado no tratamento COM_IA.
- pytest 9.1.1 - testes automatizados de aceitação.
- Radon 6.0.1 - complexidade ciclomática, LOC e Maintainability Index.
- jscpd - detecção de duplicação de código.
- Pandas, Matplotlib e Seaborn - consolidação dos dados e geração do dashboard.
- scipy.stats - teste de Wilcoxon e correlação de Spearman.
- GitHub Projects (v2) - Kanban, responsáveis, rastreabilidade e snapshots.

3.5 Tabela de Métricas

RQ	Métrica	Definição operacional	Unidade / Fonte
RQ1	Time-to-green	Tempo até todos os testes passarem; 35 min se censurado	min / script próprio
RQ2	Taxa de sucesso	testes passando / total × 100	% / pytest
RQ2	Testes falhando	Quantidade ao final do time-box	contagem / pytest
RQ3	CC média	Complexidade ciclomática média por função	CC / Radon
RQ3	Duplicação	Percentual de linhas duplicadas	% / jscpd
Controle	LOC	Linhas físicas de implementação	linhas / Radon
Extra	Maintainability Index	Índice composto de manutenibilidade	índice / Radon
Extra	Time-to-first-pass	Tempo até o primeiro teste passar	min / script próprio
Extra	Iterações e prompts	Salvamentos detectados e mensagens ao Copilot	contagem / script próprio

3.6 Inovações Propostas pelo Grupo (30% da nota)

O grupo ampliou a instrumentação prevista no enunciado para observar não apenas o resultado final, mas também a dinâmica do processo. O time-to-first-pass mede quando surge o primeiro resultado funcional; a quantidade de iterações e execuções de testes representa o número de ciclos de edição; e a quantidade de prompts caracteriza a intensidade de interação com o assistente.

Na dimensão estrutural, o Maintainability Index foi incorporado como medida complementar a complexidade, LOC e duplicação. Por fim, uma matriz de correlação de Spearman foi gerada para investigar relações entre as métricas estruturais, sem interpretar essas associações como causalidade.

4. Resultados

4.1 Coleta de Dados

A execução final registrou 12 trials válidos: 3 participantes × 4 katas, sendo 6 COM_IA e 6 SEM_IA. Cada kata possuía 10 testes de aceitação. Todos os trials terminaram antes de 35 minutos, nenhum foi censurado e todos atingiram 10/10 testes passando.

Os dados de tempo e qualidade funcional foram armazenados em dados/resultados_trials.csv. As métricas estruturais foram armazenadas em dados/metricas_estaticas.csv. As soluções finais foram preservadas por participante, kata e tratamento, permitindo reexecução das medições.

4.2 Visualização Gráfica

RQ1 - O uso de IA reduz o tempo necessário para resolver a tarefa?

Considerando diretamente os seis trials de cada tratamento, a mediana do time-to-green foi de 0,34 minuto no COM_IA e 1,50 minuto no SEM_IA, uma redução descritiva de 77,3%. Entretanto, para respeitar o desenho pareado, o Wilcoxon foi aplicado sobre a média dos dois katas de cada tratamento por participante. Com n=3 pares, o p-valor foi 0,2500, não permitindo rejeitar H0 a 5%. O Cliff's delta foi -0,556, classificado como grande e com direção favorável ao COM_IA.

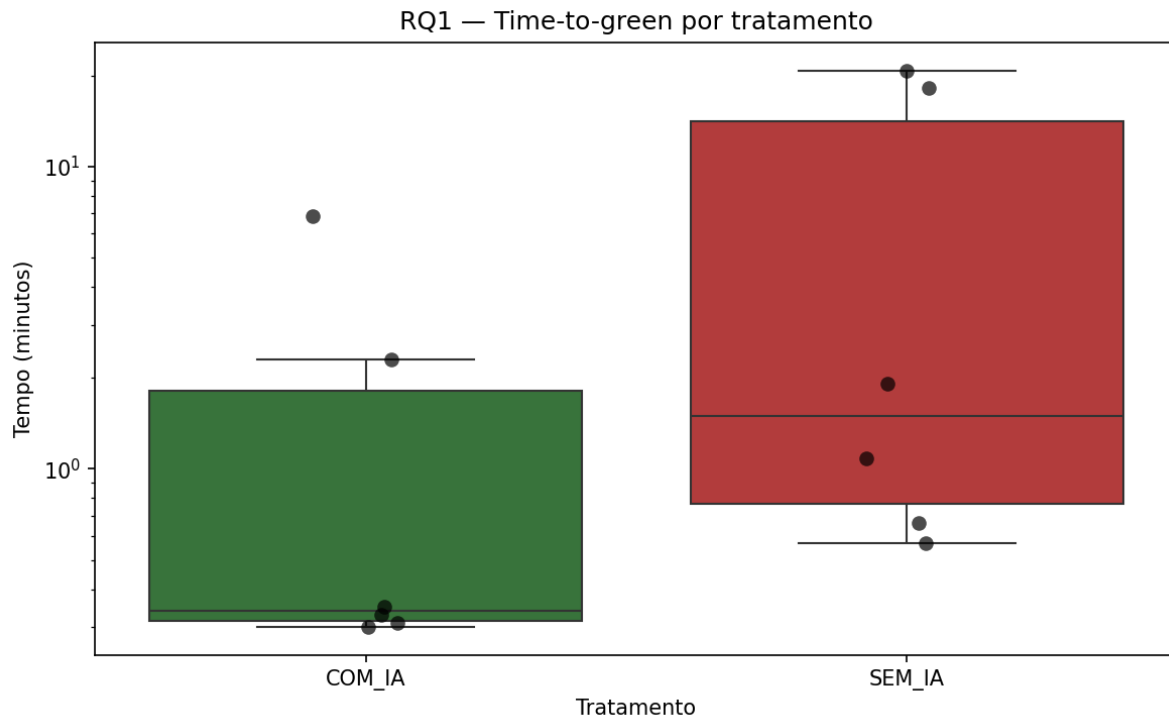


Figura 2 - RQ1: distribuição do time-to-green por tratamento.

RQ2 - O uso de IA reduz a quantidade de defeitos?

A taxa de sucesso foi 100% em todas as 12 execuções e nenhum teste permaneceu falhando ao final. A mediana foi 100% nos dois tratamentos, com IQR igual a zero. O Wilcoxon resultou em $p=1,0000$ e Cliff's delta igual a 0. Assim, a RQ2 não diferenciou os tratamentos: os katas e o time-box permitiram que todas as soluções atingissem sucesso funcional completo.

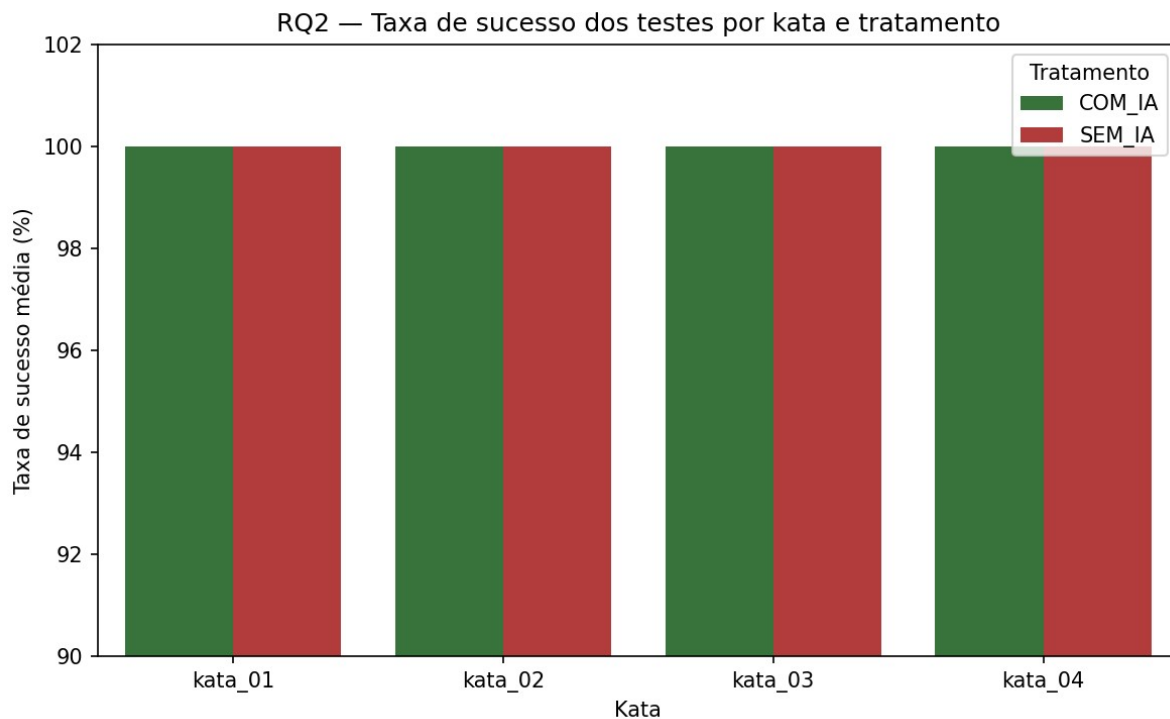


Figura 3 - RQ2: taxa de sucesso média por kata e tratamento.

RQ3 - O uso de IA altera a estrutura do código?

Nos valores diretos por trial, a mediana de complexidade ciclomática foi 10,0 no COM_IA e 11,0 no SEM_IA. A mediana de LOC foi 39,5 contra 38,5, indicando que a IA não produziu código sistematicamente mais verboso. O Maintainability Index apresentou mediana de 54,46 no COM_IA e 51,88 no SEM_IA. A duplicação foi 0% em todos os trials, portanto não houve variabilidade suficiente para comparar essa métrica.

Na análise inferencial pareada por kata, os p-valores foram 0,7500 para complexidade, 1,0000 para duplicação, 0,8750 para Maintainability Index e 0,8750 para LOC. Nenhuma diferença foi estatisticamente significativa. O MI apresentou Cliff's delta de +0,500 na convenção adotada pelo script e, nas comparações pareadas, 3 de 4 katas tiveram valor maior em COM_IA; o resultado deve ser tratado como exploratório devido ao tamanho amostral.

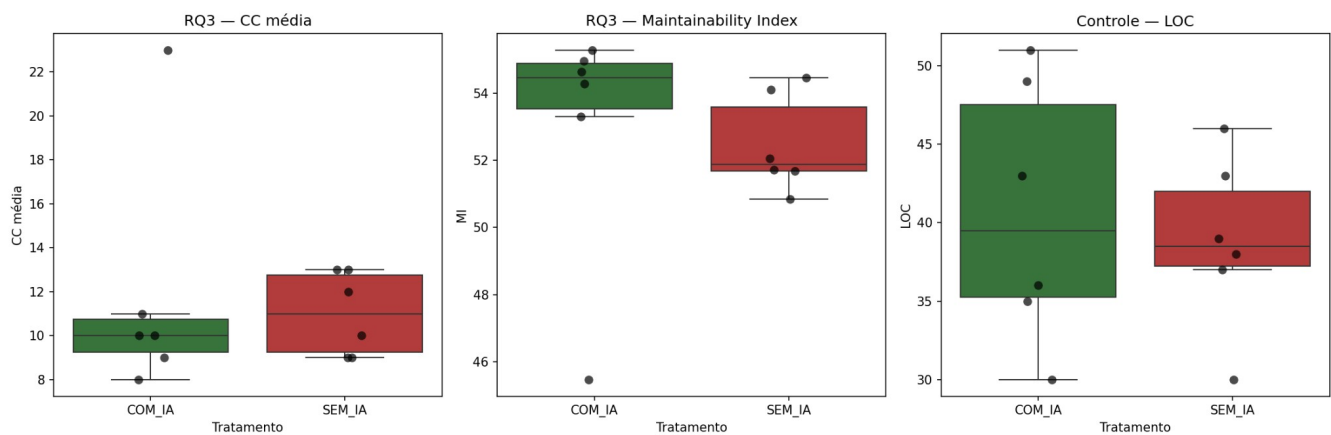


Figura 4 - RQ3: complexidade ciclomática, Maintainability Index e LOC por tratamento.

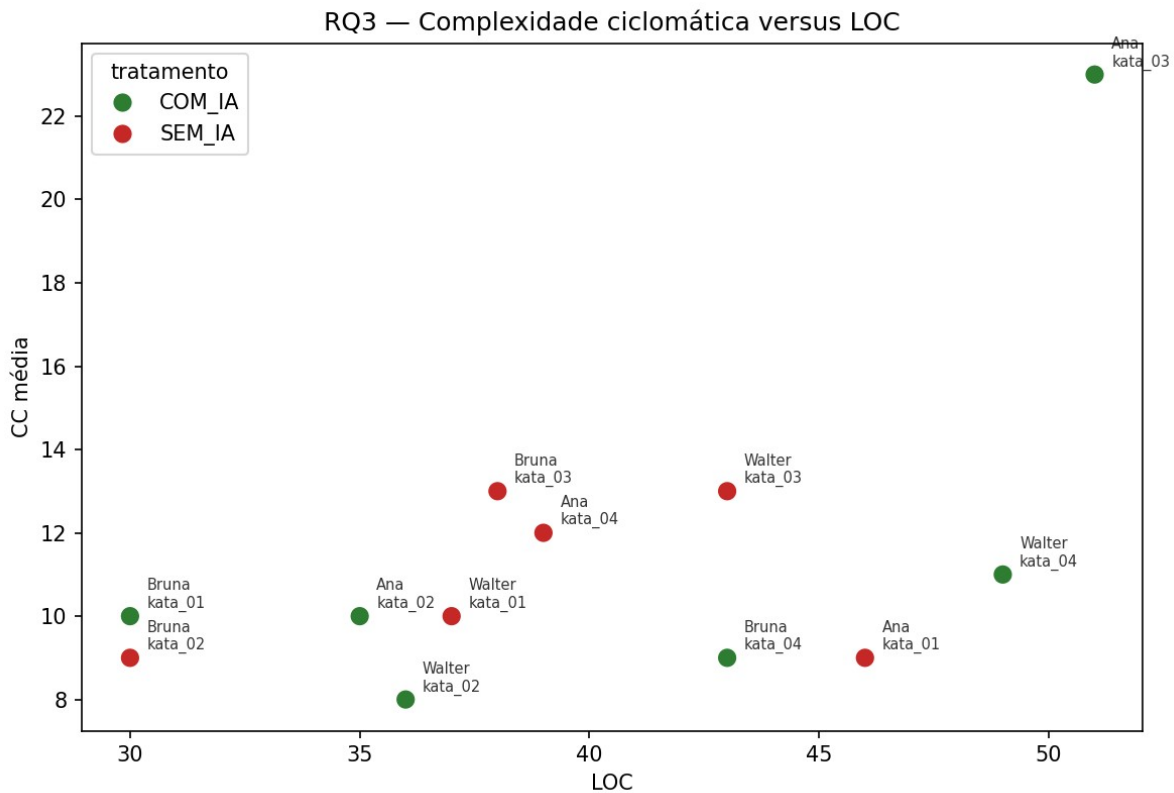


Figura 5 - RQ3: complexidade ciclomática versus LOC, com identificação dos trials.

Resultados das inovações (30%)

O time-to-first-pass apresentou mediana de 0,34 minuto no COM_IA e 1,50 minuto no SEM_IA. Nos trials com IA, a mediana de iterações de código foi 1 e a mediana de execuções automáticas de testes foi 2. No SEM_IA, as medianas foram 1,5 e 2,5, respectivamente, com dispersão muito maior causada sobretudo pelos trials manuais da Bruna. Os seis trials COM_IA utilizaram 7 prompts no total, com mediana de 1 prompt por execução.

A correlação de Spearman entre métricas estáticas mostrou associação moderada positiva entre LOC e CC média ($\rho=0,44$), associação moderada negativa entre LOC e MI ($\rho=-0,45$) e associação negativa forte entre CC média e MI ($\rho=-0,76$). A duplicação não pôde ser correlacionada porque permaneceu em 0% em todas as observações.

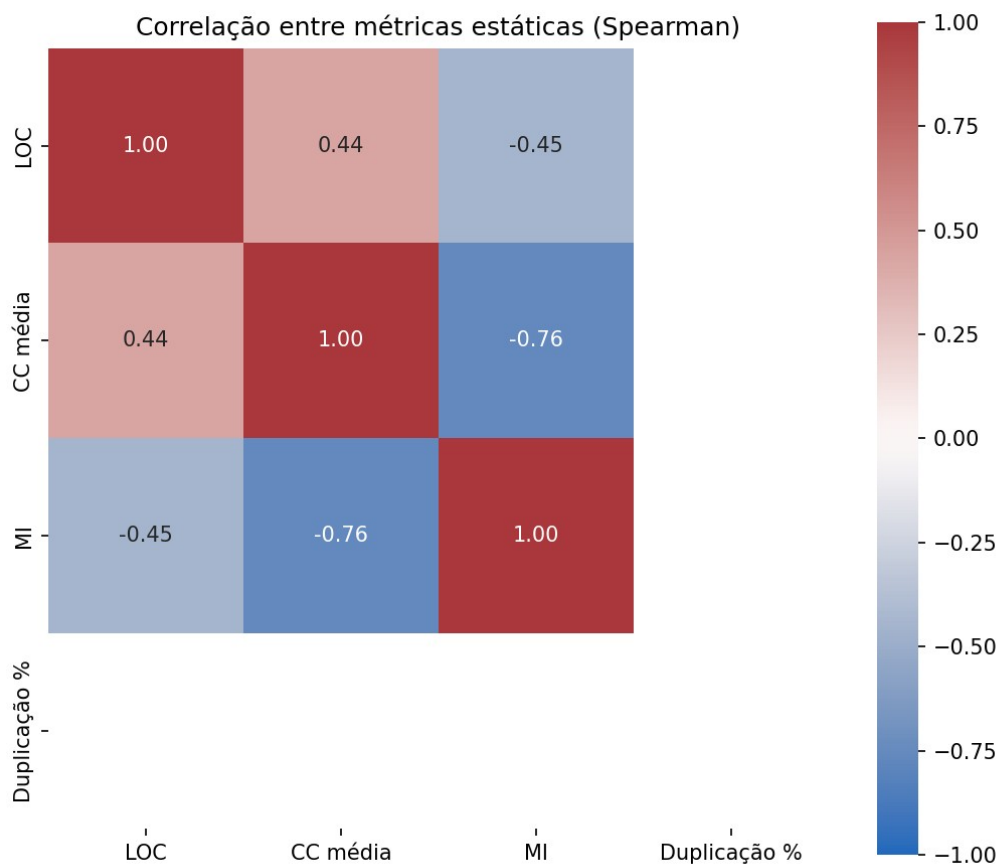


Figura 6 - Correlação de Spearman entre as métricas estruturais.

4.3 Discussão

RQ1: os dados descritivos favorecem o uso de IA, com mediana de tempo substancialmente menor no COM_IA. Porém, o teste de Wilcoxon não atingiu significância estatística. Isso não contradiz a diferença observada: com apenas três pares por participante, o menor p-valor bilateral possível é 0,25, portanto o experimento não possui poder suficiente para demonstrar significância a 5%. O tamanho de efeito e a direção consistente dos pares são mais informativos neste cenário.

RQ2: a hipótese de redução de defeitos não pôde ser sustentada nem refutada de forma informativa, pois ambos os tratamentos atingiram 100% de sucesso. O resultado sugere que, para katas pequenos e com 35 minutos disponíveis, a qualidade funcional final foi equivalente.

RQ3: a estrutura do código permaneceu globalmente semelhante entre os tratamentos. Não houve duplicação, LOC ficou próximo e a complexidade apresentou medianas semelhantes. O COM_IA apresentou MI descritivamente maior e menor CC/LOC na análise complementar, mas essas diferenças não foram significativas. O outlier de CC 23 no kata_03 da Ana mostra que o uso da IA não garante menor complexidade em todas as situações.

As métricas extras sugerem uma diferença de processo: os trials com IA geralmente chegaram ao primeiro sucesso e à solução completa com menos ciclos de edição. Entretanto, os resultados devem ser interpretados com cautela, pois participantes tinham experiências diferentes, o modelo do Copilot estava em Auto, a amostra é pequena e os katas são curtos e de domínio simples.

As principais ameaças à validade incluem efeito de aprendizado, familiaridade desigual com IA, possível variação do modelo subjacente no modo Auto, equivalência imperfeita dos katas, instrumentação por salvamentos e baixa validade externa. O contrabalanceamento, a preservação das soluções, o mesmo assistente e o protocolo comum reduziram parte dessas ameaças, mas não as eliminam.

5. Conclusão

O Lab02 executou um experimento controlado com 12 tarefas de programação para comparar GitHub Copilot e codificação manual. Todos os trials atingiram 100% de sucesso funcional, permitindo comparar produtividade e estrutura sem perda de observações por censura.

Para RQ1, o tratamento COM_IA apresentou vantagem descritiva expressiva no tempo: mediana de 0,34 minuto contra 1,50 minuto no SEM_IA. O Wilcoxon não identificou significância estatística ($p=0,2500$), resultado compatível com o baixo poder do estudo. Para RQ2, não houve diferença, pois todos os trials terminaram com 100% dos testes passando. Para RQ3, complexidade, LOC e duplicação não apresentaram diferenças significativas; o Maintainability Index foi descritivamente maior no COM_IA, mas também sem significância.

As análises extras reforçam que a principal diferença observada esteve no processo de resolução: com IA, o primeiro sucesso ocorreu mais cedo e foram necessárias menos iterações na maior parte dos trials. A correlação estrutural também mostrou que maior complexidade se associa a menor manutenibilidade na amostra. Esses resultados são exploratórios e não devem ser generalizados além do contexto estudado, mas fornecem evidência reproduzível e rastreável para discutir o efeito de assistentes de IA em tarefas curtas de programação.

6. Referências

GitHub. GitHub Copilot Documentation. <https://docs.github.com/en/copilot>.

GitHub. About Projects. <https://docs.github.com/en/issues/planning-and-tracking-with-projects>.

Pytest Documentation. <https://docs.pytest.org/>.

Radon Documentation. <https://radon.readthedocs.io/>.

jscpd - Copy/Paste Detector. <https://github.com/kucherenko/jscpd>.

SciPy. Statistical functions - scipy.stats. <https://docs.scipy.org/doc/scipy/reference/stats.html>.

ZUSE, Horst. A framework of software measurement. Walter de Gruyter, 2013.